Базы данных

A7. Анализ данных на SQL: соединения JOIN

Московский государственный технический университет
имени Н.Э. Баумана

Факультет ИБМ

Апр 2025 года

Москва

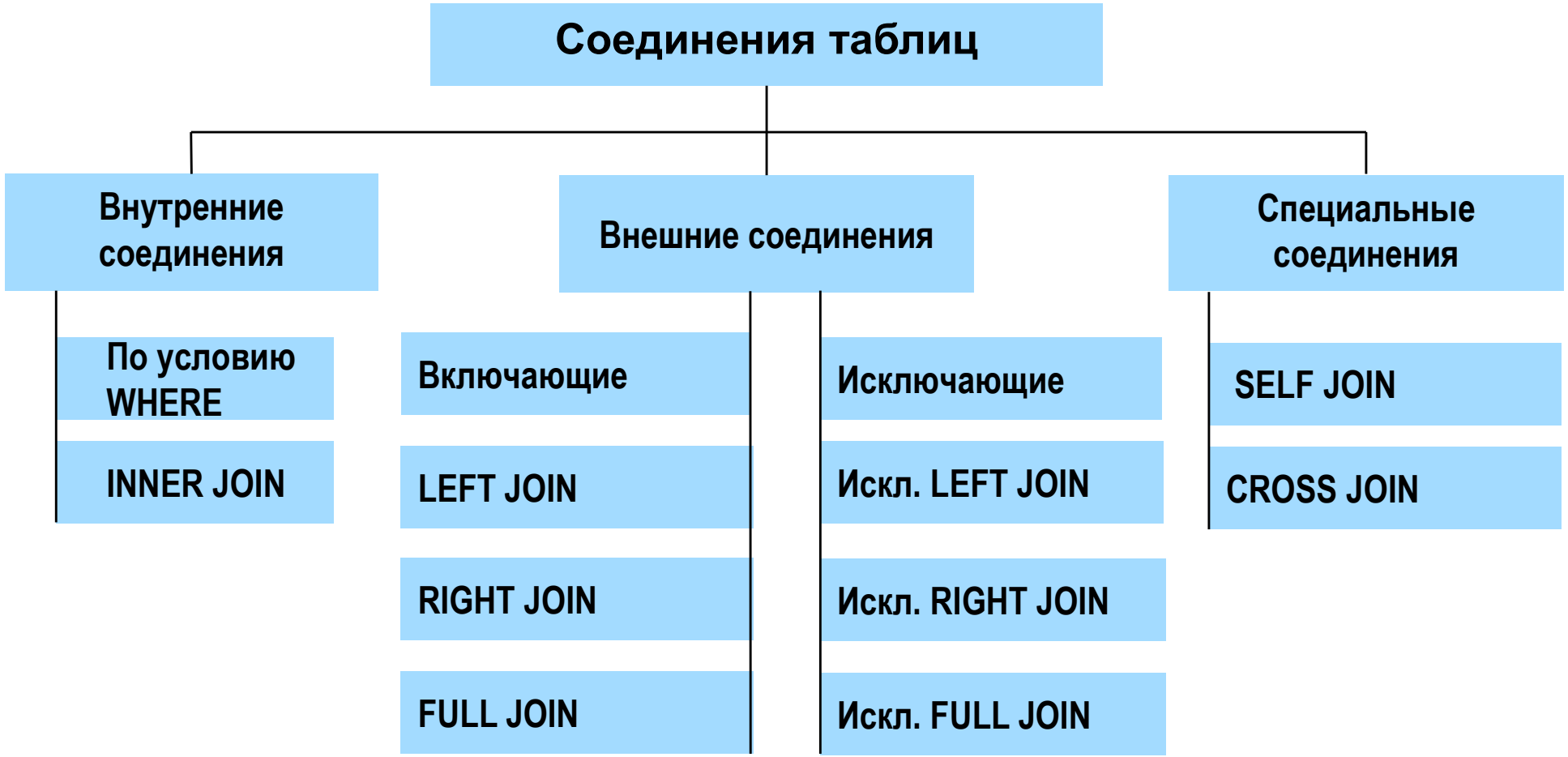
Артемьев Валерий Иванович © 2025

Соединение таблиц JOIN

Инструкции **JOIN** *соединяют* две и более *таблиц по согласованным связям* , являются основой много-табличных представлений (запросов),обеспечивают добавление столбцов, расшифровку справочных кодов, очистку и обогащение данных.

SELECT *список полей из одной или нескольких таблиц*
FROM *таблица1*
Тип соединения* JOIN *таблица2* ON *условие соединения 1
…
WHERE *общие условия отбора записей*
ORDER BY *порядок сортировки*

Классификация соединений таблиц JOIN

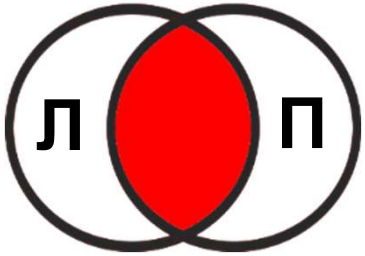


Применение разных видов соединений

- Когда нужны *только данные двух таблиц с общим ключом*, применяют **INNER JOIN** или просто **JOIN**.
- Если нужен *полный список записей одной из таблиц, объединенных с данными из другой*, используют операторы **LEFT JOIN** или **RIGHT JOIN**.
- Если нужен *полный список записей обеих таблиц*, где некоторые записи объединены, применяют оператор **FULL JOIN**.
- Если нужно *декартово произведение двух таблиц*, используют оператор **CROSS JOIN**.
- Чтобы *соединить данные из одной и той же таблицы* между собой — нужен **SELF JOIN**.

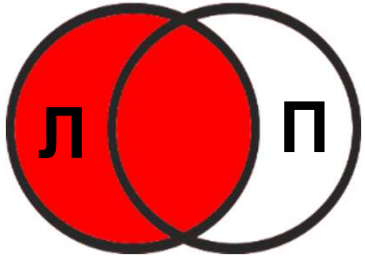
Диаграмма Венна для соединений

Inner Join



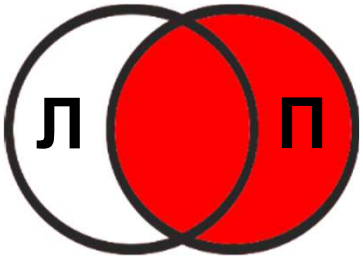

```
SELECT * FROM Л
      INNER JOIN П
ON Л.ключ=П.ключ
```

Left Join



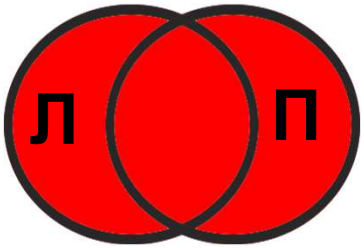

```
SELECT * FROM Л
      LEFT JOIN П
ON Л.ключ=П.ключ
```

Right Join



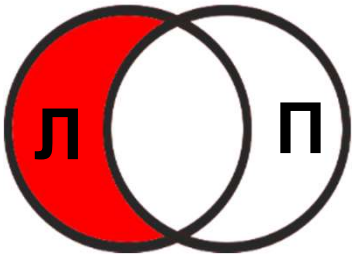

```
SELECT * FROM Л
      RIGHT JOIN П
ON Л.ключ=П.ключ
```

Full Join



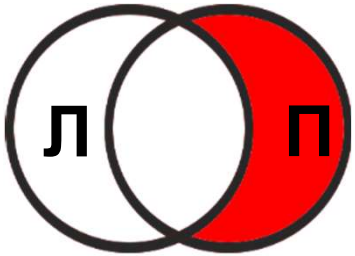

```
SELECT * FROM Л
      FULL JOIN П
ON Л.ключ=П.ключ
```

Искл. Left Join



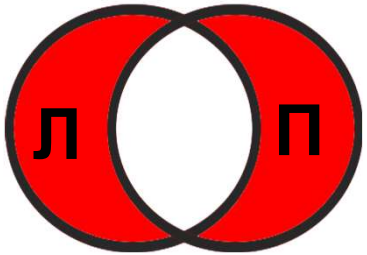

```
SELECT * FROM Л
      LEFT JOIN П
      ON Л.ключ=П.ключ
WHERE Л.ключ IS NULL
```

Искл. Right Join




```
SELECT * FROM Л
      RIGHT JOIN П
      ON Л.ключ=П.ключ
WHERE П.ключ IS NULL
```

Искл. Full Join




```
SELECT * FROM Л
      FULL JOIN П
      ON Л.ключ=П.ключ
WHERE Л.ключ IS NULL
      OR П.ключ IS NULL
```

Тестовая БД для JOIN

Persons (Сотрудники)

id_person	name	position_ref
1	Владимир	1
2	Татьяна	2
3	Александр	6
4	Борис	2

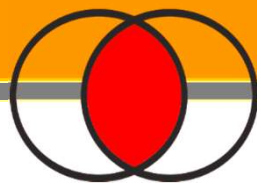
Столбец *position_ref* это ссылка на справочник должностей

Positions (должности)

id_pos	title
1	Дизайнер
2	Редактор
3	Программист

Справочник должностей

Внутреннее соединение INNER JOIN



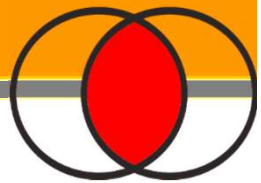
```
SELECT Persons.id_person, Persons.name,
       Positions.id_pos, Positions.title
  FROM Persons
INNER JOIN Positions
       ON Persons.position_ref = Positions.id_pos;
```

id_person	name	id_pos	title
1	Владимир	1	Дизайнер
2	Татьяна	2	Редактор
4	Борис	2	Редактор

Такое соединение покажет нам данные из таблиц, только если условие связывания соблюдается — т.е. для сотрудника указан существующий в справочнике идентификатор должности.

Если поменять порядок соединения таблиц — получим тот же результат.

Внутреннее соединение по условию WHERE

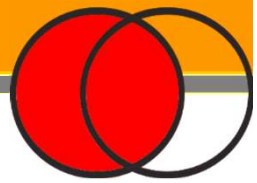


```
SELECT Persons.id_person, Persons.name,
       Positions.id_pos, Positions.title
  FROM Persons, Positions
 WHERE Persons.position_ref = Positions.id_pos;
```

id_person	name	id_pos	title
1	Владимир	1	Дизайнер
2	Татьяна	2	Редактор
4	Борис	2	Редактор

Эквивалентно INNER JOIN, даёт аналогичный результат.

Внешнее левое соединение LEFT JOIN

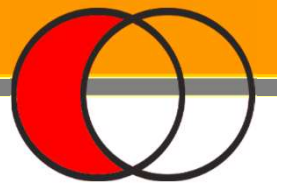


```
SELECT Persons.id_person, Persons.name,
       Positions.id_pos, Positions.title
  FROM Persons
LEFT JOIN Positions
     ON Persons.position_ref = Positions.id_pos ;
```

id_person	name	id_pos	title
1	Владимир	1	Дизайнер
2	Татьяна	2	Редактор
4	Борис	2	Редактор
3	Александр	*NULL*	*NULL*

Внешнее соединение включает в себя результаты запроса INNER и добавляются «неиспользованные» строки из правой таблицы.

Левое подмножество соединения Исключающий LEFT JOIN

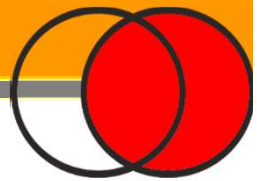


```
SELECT Persons.id_person, Persons.name,
       Positions.id_pos, Positions.title
  FROM Persons
LEFT JOIN Positions
    ON Persons.position_ref = Positions.id_pos
 WHERE Positions.id_pos IS NULL;
```

id_person	name	id_pos	title
3	Александр	*NULL*	*NULL*

Этот пример показывает, как исключить пересечение и получить только левую часть множества.
LEFT JOIN ограничиваем проверкой, что данных из 2-ой таблицы нет.
Получим тех персон, у которых нет должности с указанным ключом.

Внешнее правое соединение RIGHT JOIN

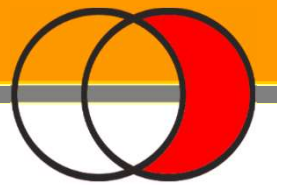


```
SELECT Persons.id_person, Persons.name,
       Positions.id_pos, Positions.title
  FROM Persons
RIGHT JOIN Positions
       ON Persons.position_ref=Positions.id_pos;
```

id_person	name	id_pos	title
1	Владимир	1	Дизайнер
2	Татьяна	2	Редактор
4	Борис	2	Редактор
NULL	*NULL*	3	Программист

Тут данные из левой таблицы присоединяются к правой. Справочник должностей (правая таблица) содержит «неиспользуемую» запись с id_pos=3 — «программист». Теперь она попала в результат запроса.

Правое подмножество соединения Исключающий RIGHT JOIN



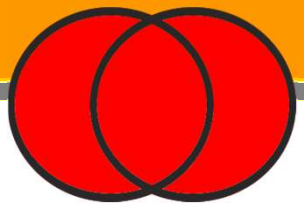
```
SELECT Persons.id_person, Persons.name,
       Persons.id_pos, Positions.title
  FROM Persons
RIGHT JOIN Positions
       ON Persons.position_ref = Positions.id_pos
WHERE Persons.id_person IS NULL;
```

id_person	name	id_pos	title
NULL	NULL	3	Программист

Пример показывает, как исключить пересечение и получить только правую часть множества RIGHT JOIN, добавив проверку, что данных из 1-ой таблицы нет.

В результате получим никем не занятые должности.

Внешнее полное соединение FULL JOIN

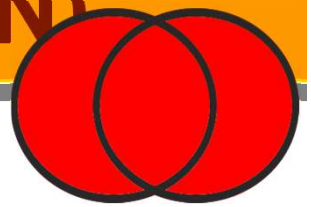


```
SELECT Persons.id_person, Persons.name,
       Positions.id_pos, Positions.title
  FROM Persons
FULL JOIN Positions
     ON Persons.position_ref = Positions.id_pos;
```

id_person	name	id_pos	title
1	Владимир	1	Дизайнер
2	Татьяна	2	Редактор
4	Борис	2	Редактор
3	Александр	*NULL*	*NULL*
NULL	*NULL*	3	Программист

В результате такого соединения получим все записи из левой и правой таблицы.

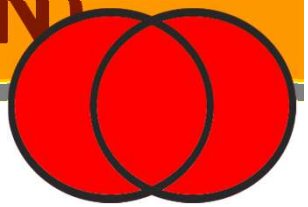
1-й способ имитации полного соединения (LEFT JOIN и RIGHT JOIN)



```
(SELECT Persons.id_person, Persons.name,
        Positions.id_pos, Positions.title
   FROM Persons
LEFT JOIN Positions
     ON Persons.position_ref = Positions.id_pos)
 UNION
(SELECT Persons.id_person, Persons.name,
        Positions.id_pos, Positions.title
   FROM Persons
RIGHT JOIN Positions
     ON Persons.position_ref = Positions.id_pos);
```

При таком вызове UNION, после слияния результатов будут отсечены дубли (как при DISTINCT). Для отсечения применяется сортировка, что может сказываться на быстродействии.

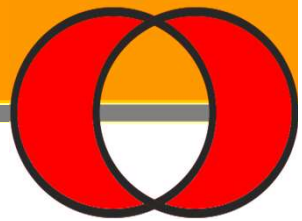
2-й способ имитации полного соединения (LEFT JOIN и RIGHT JOIN)



```
(SELECT     Persons.id_person, Persons.name,
            Positions.id_pos, Positions.title
FROM        Persons
LEFT JOIN   Positions
        ON  Persons.position_ref = Positions.id_pos)
UNION ALL
(SELECT     Persons.id_person, Persons.name,
            Positions.id_pos, Positions.title
FROM        Persons
RIGHT JOIN  Positions
        ON  Persons.position_ref = Positions.id_pos)
WHERE       Persons.id_person IS NULL);
```

Объединение LEFT и RIGHT, но в одном из запросов мы исключаем часть, соответствующую INNER. Используем UNION ALL, что позволяет обойтись без сортировки.

Исключающее полное соединение
Исключающий FULL JOIN

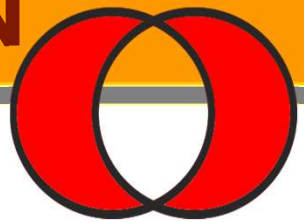


```
SELECT Persons.id_person, Persons.name,
       Positions.id_pos, Positions.title
  FROM Persons
FULL JOIN Positions
    ON Persons.position_ref = Positions.id_pos
 WHERE Persons.id_pos IS NULL
    OR Persons.id_person IS NULL
```

id_person	name	id_pos	title
3	Александр	NULL	NULL
NULL	NULL	3	Программист

Всё, кроме пересечения – этот запрос соберет все случаи, когда по какой-то причине данные из таблиц не связаны.

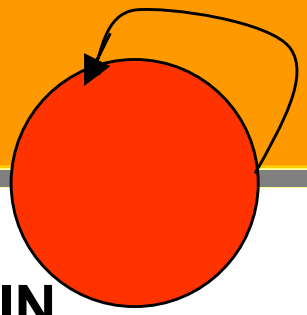
Имитация исключающего FULL JOIN



```
(SELECT Persons.id_person, Persons.name,
        Persons.id_pos, Positions.title
   FROM Persons
 LEFT JOIN Positions
        ON Persons.position_ref = Positions.id_pos
  WHERE Persons.id_pos IS NULL)
UNION ALL
(SELECT Persons.id_person, Persons.name,
        Persons.id_pos, Positions.title
   FROM Persons
RIGHT JOIN Positions
        ON Persons.position_ref = Positions.id_pos
  WHERE Persons.id_person IS NULL)
```

Исключающее пересечение множеств FULL JOIN можно получить, сложив левое и правое подмножества соединений через UNION ALL (т.к. подмножества не пересекаются).

Соединение SELF JOIN



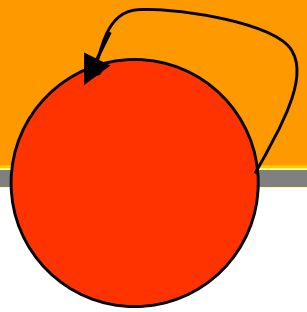
Выполняется соединение данных из одной и той же таблицы. В SQL нет отдельного оператора для **SELF JOIN** соединения, поэтому пользуйтесь операторами разными видами **JOIN** или **WHERE**. Чтобы различать оригинальную таблицу и её копию, применяйте алиасы.

```
SELECT А.поле1, Б.поле2
  FROM Таблица AS А
   JOIN Таблица AS Б
     ON А.общее_поле = Б.общее_поле;
```

Используется для решения различных аналитических задач:

- нужно сравнить строки в одной и той же таблице
- извлечь данные из разных строк, которые связаны между собой.
- объединение записей внутри одной таблицы.
- развёртка оргструктуры: найти сотрудников каждого руководителя,
- анализ временных рядов данных.

Пример SELF JOIN: Менеджеры и их подчинённые

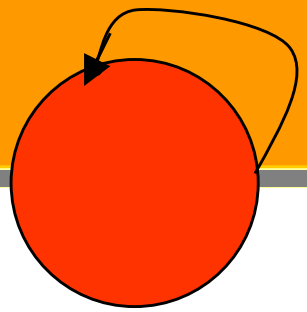


Имеется таблица о сотрудниках СОТРУДНИКИ с указанием руководителя (руководитель_id).
Нужно найти всех сотрудников, которые работают под одним и тем же руководителем.
Соединим таблицу СОТРУДНИКИ с самой собой, и затем отфильтруем результаты, чтобы исключить совпадения одного и того же сотрудника.

```
SELECT А.сотрудник_ФИО AS Сотрудник1,
       Б.сотрудник_ФИО AS Сотрудник2
  FROM Сотрудники AS А
   JOIN  Сотрудники AS Б
     ON А.руководитель_id = Б.руководитель_id
WHERE А. сотрудник_id <> Б. сотрудник_id;
```

Получим пары сотрудников, работающих под одним и тем же руководителем.

Пример SELF JOIN: Иерархические структуры

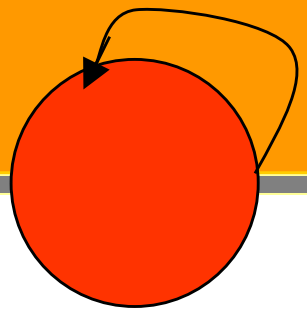


SELF JOIN можно использовать для работы с иерархиями данных, такими как оргструктуры или каталоги продуктов. Пусть имеется таблица КАТЕГОРИИ с информацией о категориях и подкатегориях продуктов, можно применить **SELF JOIN** для извлечения всех подкатегорий для каждой категории.

```
SELECT A.категория_имя AS Категория,
       B.категория_имя AS Подкатегория
  FROM КАТЕГОРИИ AS A
    JOIN КАТЕГОРИИ AS B
      ON A.категория_id = B.родит_категория_id;
```

Это позволяет нам анализировать иерархические структуры данных и выявлять взаимосвязи между различными уровнями иерархии.

Пример SELF JOIN: Нахождение предыдущей записи



Имеется таблица ЗАКАЗЫ с информацией о заказах, включая ID заказа и дату заказа. Нужно найти предыдущий заказ для каждого заказа. Это полезно для анализа последовательности заказов и выявления тенденций в поведении клиентов.

Используем **SELF JOIN**, чтобы соединить таблицу ЗАКАЗЫ с самой собой, и находим предыдущий заказ для каждого заказа на основе даты заказа.

```
   SELECT A.заказ_id, A.заказ_дата,
          B.заказ_id AS Предыд_заказ_ID,
          B.order_date AS Предыд_заказ_дата
     FROM ЗАКАЗЫ AS A
LEFT JOIN ЗАКАЗЫ AS B
       ON A.заказ_дата > B.заказ_дата
    WHERE B.заказ_дата IS NOT NULL
ORDER BY A.заказ_дата;
```

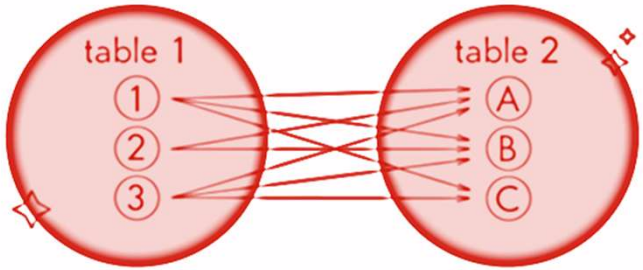
Позволяет анализировать временные ряды данных и выявлять закономерности в поведении клиентов, что полезно для оптимизации бизнес-процессов и улучшения клиентского опыта.

Перекрёстное соединение CROSS JOIN

Оператор соединения, создающий **декартово произведение** двух таблиц. Прекрасно подходит для генерации всех возможных **комбинаций** данных из этих таблиц.

Порождает **большое число записей**, равное произведению количества записей соединяемых таблиц.

Чтобы исключить "Чернобыль данных", нужно использовать **механизмы фильтрации**.



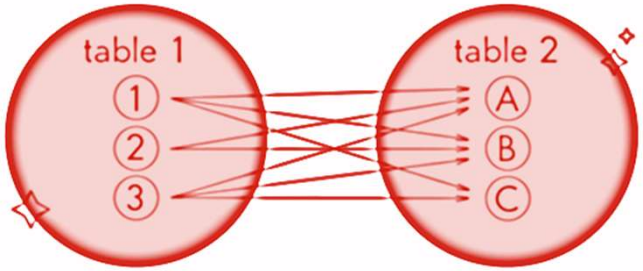
Используется:

- при разработке **характеристик продуктов**
- для составления **расписаний игровых турниров**
- для создания **многоязычных словарей**
- при разработке подробного **плана техобслуживания оборудования**, комбинируя его с требуемыми проверками и процедурами.
- для генерации **справочника календаря**
- для генерации **тестовых данных,** покрывающих все возможные сценарии, что помогает выявлять дефекты.

Пример CROSS JOIN: генерация характеристик продукта

Для примера сгенерируем сочетания цветов и размеров футболок, соединив перекрёстно таблицы цветов и размеров футболок:

```
SELECT Футволки_Цвета.цвет          AS Цвет_футболки,
       Футболки_Размеры.размер AS Размер_футболки
  FROM Футволки_Цвета
CROSS JOIN Футболки_Размеры;
```



Или аналогичный вариант:

```
SELECT Футволки_Цвета.цвет          AS Цвет_футболки,
       Футболки_Размеры.размер AS Размер_футболки
  FROM Футволки_Цвета, Футболки_Размеры;
```

Цвет_футболки	Размер_футболки

Терпения и удачи всем, кто связан с базами данных

Спасибо за внимание!

Валерий Иванович Артемьев

МГТУ имени Н.Э. Баумана, кафедра ИУ-5

Банк России
Департамент данных, проектов и процессов

Тел.: +7(495) 753-96-25
e-mail: viart@bmstu.ru